

The background of the top section is a dark blue field filled with various white icons representing technology and entertainment, including smartphones, tablets, cameras, game controllers, and speakers. Three large, stylized chevrons in blue, yellow, and grey point to the right, positioned above the main title.

Innovative AI-Enabled Hackathon

Basic Guide for Star, Supernova and Galaxy



Introducing the AI-Enabled Hackathon

THE STORY CONINUES...

Welcome Back, STRIPE Innovators!

For decades, Ride Rush Amusement Park has been the heartbeat of our city's joy and excitement. Your earlier innovations transformed it into a smarter, data-driven experience—where every ride, every guest, and every moment could be measured, understood, and improved. The Park Operations Simulator proved that, with the right code, data could truly come to life.

But now, as Ride Rush prepares for the STRIPE International Competition, the management has realized something critical:

“Seeing the data is no longer enough—they must respond to it in real time.”

CHALLENGE INTRODUCTION

Ride Rush Park operates in a constantly changing environment, and on this day, weather becomes the most critical challenge. Rain showers, sudden wind gusts, and rising temperature levels continuously affect how rides can operate. These conditions can cause rides to run normally, slow down, or shut down entirely due to safety concerns.

Your task is to design and build a Weather Impact Decision Engine that simulates these changing conditions and determines how each ride is affected in real time. The system should help park operators understand what is happening at every hour and make informed decisions.

Your solution must simulate a full day, automatically evaluate ride conditions, and present the system clearly through a structured interface. The goal is to build a system that is reliable, responsive, and easy to understand.

Solution Requirements

1. Simulation Engine (12-Hour Day)

- Simulate a full park day from 10:00 AM to 9:00 PM (10 to 21 hours)
- Use a 10-second clock where 10 real seconds = 1 simulation hour
- Structure your program so that each loop/iteration represents one hour
- Implement a delay so that the system updates every 10 seconds
- At each hour, the system must automatically:
 - Generate new weather values
 - Recalculate ride impacts
 - Update the display

2. Weather Conditions (Random Generation)

- Generate the following values using random number generation every hour:

Condition	Minimum	Maximum
Rainfall Intensity	0	10
Wind Speed	0	60
Temperature	18	45

- Change every hour
- Stay within the defined ranges
- Represent realistic weather variation

3. Weather Impact Levels

- Convert generated values into impact levels using the ranges below:

Condition	Low Impact	Moderate Impact	High Impact
Rainfall Intensity	0–3	4–6	7–10
Wind Speed	0–20	21–40	41–60
Temperature	18–25	26–32	33–45

- Map impact levels to ride states:
 - Low Impact → FULL (fully operational)
 - Moderate Impact → SLOW (reduced performance or capacity)
 - High Impact → STOP (unsafe, not operational)

Solution Requirements

4. Ride System (6 Required Rides)

- Include all of the following rides in your system:
 - Lazy River
 - Nebula Spinner
 - Pixel Arcade
 - Rocket Slingshot
 - Splashing Mountain
 - Titan Coaster
- Each ride must:
 - Be evaluated every hour
 - Respond to one or more weather conditions
 - Update its status dynamically

5. Ride Impact Logic

- Use the following mapping to determine ride behavior:

Ride	Rain (Moderate)	Rain (High)	Wind (Moderate)	Wind (High)	Temp (Moderate)	Temp (High)
Lazy River	SLOW	STOP	FULL	SLOW	FULL	SLOW
Nebula Spinner	SLOW	SLOW	SLOW	STOP	FULL	FULL
Pixel Arcade	FULL	FULL	FULL	FULL	FULL	FULL
Rocket Slingshot	STOP	STOP	STOP	STOP	FULL	FULL
Splashing Mountain	SLOW	STOP	SLOW	STOP	FULL	SLOW
Titan Coaster	SLOW	STOP	SLOW	STOP	FULL	SLOW

- Apply the following rules:
 - A ride may be affected by multiple conditions at the same time
 - The most severe impact [STOP > SLOW > FULL] determines the final ride state
 - Update ride states at every hour based on new inputs

Solution Requirements

6. Decision Logic

- Your system must:
 - Use loops to run continuously
 - Use conditional logic (if/else) to interpret values
 - Convert raw weather values into impact levels
 - Apply ride-specific logic rules
 - Combine multiple conditions correctly
 - Update outputs dynamically over time
- Your logic should clearly show:
 - How inputs are processed
 - How decisions are made
 - How outputs are generated

7. User Interface (Required)

- Display the following clearly at every time step:
 - Current hour (10–21)
 - Rainfall intensity
 - Wind speed
 - Temperature
 - Status of all 6 rides
- For each ride, display:
 - Ride name
 - Visual representation (label or image)
 - Current state: FULL / SLOW / STOP
- Ensure the interface:
 - Updates every hour
 - Is structured and easy to read
 - Clearly reflects changes in the system

Coding Guidelines for all Categories

To successfully complete this challenge, students should be familiar with several core programming concepts. These skills will enable them to transform system data and dynamic inputs into intelligent, working solutions. Below is a breakdown of the key coding skills required.

1. Variables & Data Types:

At the foundation, students must understand how to work with variables and data types, including strings, integers, and floating-point values, in order to store and manage information effectively.

In advanced categories, they should also be comfortable using more advanced structures such as arrays, lists, and/or dataframes [tables] to group related data, and dictionaries or maps to connect values logically, enabling efficient data organization within their system.

2. Conditional Statements:

Logic is key! Students must be able to use if, else if, and else statements to make decisions based on the data.

- Example: If wait_time is greater than 30 (minutes), then suggest a different ride.

3. Loops

The students must understand how to use loops, such as for and while, to process repeated actions, iterate through datasets, and power time-based simulations where system states update continuously.

4. Functions:

Students are also expected to write functions to organize their code into reusable, structured components. This not only improves readability but allows them to apply the same logic—such as calculations or rule checks—efficiently throughout their program.

5. Random Numbers and File Reading:

A key element of this challenge is the use of random number generation and file reading, which together create dynamic and evolving scenarios. Students must generate random values to simulate unpredictable conditions and read structured data from files to drive system behavior, ensuring their solutions adapt instead of relying on fixed inputs.

6. UI elements:

Students will need to use a UI library if their coding language of choice does not have a built in way to generate their visualization of the park.

Recommended Skill Levels by Category

To ensure a fair and engaging experience for all participants, the hackathon is designed with progressive skill expectations across categories. Each level builds on the core programming concepts, encouraging students to deepen their understanding and apply more advanced techniques as they move up. The following table outlines the recommended skill levels, helping teams understand the complexity of logic, data handling, and system design expected for their category.

Concept	Star	Super Nova	Galaxy
Variables	Basic (Strings/Ints/ Floats / Booleans)	Arrays/Lists	Dictionaries/ DataFrames / Tables
Data Input	Random Numbers / Hard-coded Values	Random Numbers / File I/O	Random Numbers / File I/O
Logic	Simple if/else	Nested Conditionals	Complex Logic Gates
Loops	Basic for loops	Loops with logic	Efficient data iteration
Function	Using built-ins	Creating functions	Functions with parameters

Note: Students may use any programming language, IDE, AI tools, or other resources they are comfortable with to solve the problem.

Project Completion Checklist

Section	Criteria	Check
Simulation & System Behavior	Simulation runs from 10:00 to 21:00 using 10-second clock	☐
	System updates automatically every hour	☐
	Weather values are randomly generated within defined ranges	☐
	Simulation runs continuously without crashing	☐
Weather Processing	Rain, wind, and temperature are generated correctly	☐
	Values are converted to impact levels correctly	☐
	Impact levels follow defined thresholds	☐
Ride Logic	All 6 required rides are included	☐
	Ride responses match the defined mapping rules	☐
	Ride states (FULL / SLOW / STOP) are assigned correctly	☐
	Multiple weather conditions are handled correctly	☐
	Most severe impact determines final ride state	☐
Decision Logic	Uses loops to simulate time progression	☐
	Uses conditional logic to determine impacts	☐
	Correctly maps inputs to outputs	☐
User Interface	Displays current time clearly	☐
	Displays all three weather conditions	☐
	Displays all 6 rides with current status	☐
	Output updates clearly at each time step	☐
Day 2 Adaptation	Integrates file-based data correctly	☐
	Implements new rule or constraint	☐
	System remains stable after updates	☐
Final Verification	System behavior matches requirements	☐
	Output reflects correct logic throughout	☐
	Team demonstrates understanding of system	☐

Challenge Rubric

Category	Criteria	Max Points
System Development & Feature Completeness	Simulation Engine (12-hour clock, 10s timing, continuous run)	30
	Weather Generation (rain, wind, temperature updated each hour)	20
	Data Structures & Logic (handling weather + rides effectively)	25
	UI / Output Clarity (clock, weather cards, ride status display)	25
Decision Logic & Problem Solving	Decision Accuracy (correct impact levels: FULL / SLOW / STOP)	40
	Dynamic Handling (multiple weather conditions per ride)	30
	Efficiency & Design (clear logic, correct prioritization of most severe impact)	30
Adaptability & Day 2 Integration	File-Based Data Integration (weather data replaces random input)	30
	Requirement Update (new rule applied to ride/weather logic)	40
	System Stability (simulation continues correctly after changes)	30
Code Quality & Technical Implementation	Code Structure & Readability (clean, modular, understandable logic)	50
Presentation & Demonstration	Presentation Quality (clear explanation of system and logic)	20
	Live Demonstration (system runs and updates correctly in real time)	15
	Understanding & Communication (explains weather logic, ride impact, decisions)	15
TOTAL		400

Program Schedule

Date	Time	Format	AI Hackathon Event	Programming Competition Schedule
12 Jun 2026	11:59 PM	Virtual (Competition Website)	AI Hackathon requirements released / Students start working on the challenge asynchronously	N/A
13 Jun 2026	2:00 PM	Virtual (Zoom)	AI Hackathon Information Session and Q&A Webinar Zoom link: https://us02web.zoom.us/j/6943630292?pwd=WkZtVGZEWklhZThDazN4SUFkOVZzQT09 Meeting: 1478	N/A
14 Jun 2026	8:30 AM – 9:30 AM	In-person	AI Hackathon: Coding and Solution Modification	Setting up and Preparation
14 Jun 2026	9:30 AM – 11:00 AM	In-person		Programming Judging and Team Presentations
14 Jun 2026	12:00 PM – 2:00 PM	In-person	AI Hackathon Judging and Team Presentations	N/A
14 Jun 2026	2:00 PM – 3:00 PM	In-person	Public Showcase	